The key goal is designing a resilient system that, in the best-case scenario, can provide its services without the user noticing that there has been a failure. In contrast, in the worst-case scenario, it should still work but with graceful degradation.

NOTE Spring WebFlux and Project Reactor are exciting subjects in the Spring landscape. If you want to learn more about how reactive Spring works, I recommend looking at *Reactive Spring* by Josh Long (https://reactivespring.io). In the Manning catalog, see part 3 of *Spring in Action*, sixth edition, by Craig Walls (Manning, 2022).

In the next section, you'll write automated tests to verify the different aspects of the Order Service application.

8.5 Testing reactive applications with Spring, Reactor, and Testcontainers

When an application depends on a service downstream, you should test the interaction against the API specification of the latter. In this section, you'll first try the `BookClient` class against a mock web server acting as the Catalog Service to ensure the correctness of the client. Then you'll test the data persistence layer with sliced tests using the `@DataR2dbcTest` annotation and Testcontainers, much as you did in chapter 5 with `@DataJdbcTest`. Finally, you'll write sliced tests for the web layer using the `@WebFluxTest` annotation, which works in the same way as `@WebMvcTest` but for reactive applications.

You already have the necessary dependencies on the Spring Boot test libraries and Testcontainers. What is missing is a dependency on `com.squareup.okhttp3:mockwebserver`, which will